Durham University

Version Control: GitHub & *Collaboration*

Chris Marcotte[1]
Durham University

[1]For errata and questions: christopher.marcotte@durham.ac.uk

Overview

Outline

Remote Repositories 3
GitHub 6
File Management 24
Collaborative Development 27
Summary 30

Overview

Outline

Remote Repositories 3
GitHub 6
File Management 24
Collaborative Development 27
Summary 30

Learning Goals

- Understanding of *Git* & GitHub integration
- Ability to use GitHub with local repositories
- Understanding of best practice

Remote Repositories

GitHub, GitLab, and *Git*ting started

There are two dominant available *Git* hosts – GitLab and GitHub.

They have their strengths and weaknesses – GitHub is more popular.[1]

To use these services, you'll need an account.[2]

[1] It is also free, which is why we will use it.

[2] Alternatively, you can self-host your own *Git* server, but this is more involved and may not be ideal for beginners.

GitHub, GitLab, and *Git*ting started

There are two dominant available *Git* hosts – GitLab and GitHub.

They have their strengths and weaknesses – GitHub is more popular.[1]

To use these services, you'll need an account.[2]

[1] It is also free, which is why we will use it.

[2] Alternatively, you can self-host your own *Git* server, but this is more involved and may not be ideal for beginners.

GitHub and *Git*

It is important to note that GitHub (or GitLab) and *Git* are not the same entity.

Git is an open-source program for version control which is based on a distributed repository model with cheap branching.

GitHub is a *Git* host and website owned by Microsoft, which is leveraging its dominant market position to sell yours and many others work back to you through the white-washing of generative AI[1].

[1] A decentralised, open, peer-to-peer version control system is Radicle.

GitHub and *Git*

It is important to note that GitHub (or GitLab) and *Git* are not the same entity.

Git is an open-source program for version control which is based on a distributed repository model with cheap branching.

GitHub is a *Git* host and website owned by Microsoft, which is leveraging its dominant market position to sell yours and many others work back to you through the white-washing of generative AI[1].

Last lecture was about *Git*, this one is about using *Git* with GitHub.

[1] A decentralised, open, peer-to-peer version control system is Radicle.


GitHub

Navigating GitHub

Creating an account

1. Navigate to github.com
2. Click the Sign up button in the upper-right corner
3. Enter an email address for your account (e.g., your Durham University email)
4. Manage your login security[1]

[1] Including whether to keep your email address private!

Creating an account

1. Navigate to github.com
2. Click the Sign up button in the upper-right corner
3. Enter an email address for your account (e.g., your Durham University email)
4. Manage your login security[1]

A GitHub account is an important marker of your professional presence.

[1] Including whether to keep your email address private!

Creating a new repository on GitHub

Creating a new repository[1] requires:

1. Name (GitHub will offer you an available but nondescript – bad! – name)
   - Description of the repository content (concise!)
2. Select *Public* or *Private* – if other GitHub users can see it
3. Initialized with:
   - README (important information for a new user)
   - a `gitignore` file (language-dependent files that *Git* will not track)
   - License (dependency-dependent!)

[1] You can also import an existing repository (though only *Git* ones, now!) into GitHub for hosting.

GitHub Website features

- User Page
  - The only subpage that's relevant for us is *Repositories*
- Repository Page
  - Issues – List of problems with the content in a repo
  - Pull requests – Proposed solutions to problems with the content in a repo
  - ~~Discussions – A built-in forum for a repository~~
  - Actions – Configuration of automated procedures for the repo
  - ~~Projects – Organizational tooling for Issues and Pull requests~~
  - ~~Security – A policy for reporting security issues responsibly~~
  - ~~Insights – Statistical perspectives on a repo~~
  - Settings – repository settings; importantly: access rules

GitHub Basics

Publicly available *Git* repositories are... *public*!

You should maintain standards of professionalism when publishing:

- Giving issues and pull-requests charitable understanding
- Using clear, concise, and professional language in your files
- Maintaining adequate documentation for reuse of your code

GitHub Basics

Publicly available *Git* repositories are... *public*!

You should maintain standards of professionalism when publishing:

- Giving issues and pull-requests charitable understanding
- Using clear, concise, and professional language in your files
- Maintaining adequate documentation for reuse of your code

Your presence on GitHub is often something employers will look at!

Interlude: Setting up `ssh` access

Local-only repos only need the *modify* → `git add` → `git commit` cycle.

The real utility of using GitHub over local *Git* is storing and sharing code.

[1] I use public-private key pairs *on my machine(s)*, but it is up to you to make an informed decision.

Interlude: Setting up `ssh` access

Local-only repos only need the *modify* → `git add` → `git commit` cycle.

The real utility of using GitHub over local *Git* is storing and sharing code.

You will mostly use *Git* in tandem with your GitHub account – i.e. with `ssh`.

Setting this up is not difficult but it does take a little time. Broadly two choices:

- `ssh-agent` forwarding (convenient, secure; limited, windows issues)
- manually creating and adding public-private-key pairs to your account[1]

[1] I use public-private key pairs *on my machine(s)*, but it is up to you to make an informed decision.

`git remote`

The command `git remote add <name> <url>` connects our *local repo* to the *remote repo* at `<url>`.

This is the general format of the command.

This establishes the connection between your local repository and the remote one on GitHub.

Having done so[1], we can run `git push` to push our local files to the remote repo.

Later on, someone else may wish to `git pull` our updates!

---

[1] Many people prefer to simply *create* the repo on GitHub from the beginning, rather than locally, and clone the empty repo to create the local workspace.

Pushing & Pulling

With `ssh` set up, we can interact with remote repositories.

`git push` and `git pull` are commands for interacting with a remote repository from your local repository.

`git push` instructs the *Git* program to *push* the commits in the current local repository to the remote counterpart. I.e., puts your local changes on GitHub.

`git pull` instructs the *Git* program to *pull* the changes from the remote repository to your local copy. I.e., puts GitHub stuff in your local repository.

Pushing & Pulling

With `ssh` set up, we can interact with remote repositories.

`git push` and `git pull` are commands for interacting with a remote repository from your local repository.

`git push` instructs the *Git* program to *push* the commits in the current local repository to the remote counterpart. I.e., puts your local changes on GitHub.

`git pull` instructs the *Git* program to *pull* the changes from the remote repository to your local copy. I.e., puts GitHub stuff in your local repository.

You need `ssh` access to a corresponding GitHub repo to use these commands.

`git fetch` & `git pull`

Getting the remote repository changes is a two-step process:

- `git fetch origin main` downloads the changes from the remote repository `origin` on branch `main` to your local repository.
- `git merge main` merges the changes from the remote repository into your local repository on branch `main`.

`git fetch` & `git pull`

Getting the remote repository changes is a two-step process:

- `git fetch origin main` downloads the changes from the remote repository `origin` on branch `main` to your local repository.
- `git merge main` merges the changes from the remote repository into your local repository on branch `main`.

In practice we combine these two steps into one with: `git pull`

By default, applies to the current branch (pulling the remote into the local).

`git push`

We run `git status` and see:

```
On branch main
nothing to commit, working tree clean
```

`git push`

We run `git status` and see:

```
On branch main
nothing to commit, working tree clean
```

If we:

1. edit the file `file1.c`, and
2. do the usual `git add` and `git commit`,
3. then we *push* the local changes to the remote with the command `git push`

`git push`

We run `git status` and see:

```
On branch main
nothing to commit, working tree clean
```

If we:

1. edit the file `file1.c`, and
2. do the usual `git add` and `git commit`,
3. then we *push* the local changes to the remote with the command `git push`

This is great when it works, but sometimes there's a **conflict** – e.g. when another person has modified the repository while you have.

Remote Repository Conflicts

We've already discussed how to use `git push` & `git pull`.

- `git push` reflects your local changes in a remote repository, and
- `git pull` updates your local repository with the changes in the remote.

But these only work in the typical way if the remote has a matching branch.

Remote Repository Conflicts

We've already discussed how to use `git push` & `git pull`.

- `git push` reflects your local changes in a remote repository, and
- `git pull` updates your local repository with the changes in the remote.

But these only work in the typical way if the remote has a matching branch.

One or the other can fail due to merge conflicts.

You can use `git pull --rebase` to *rebase* your local repository on the remote.

Confusing *Git* terminology

Julia Evans[1] has spent considerable time using, thinking about, and cataloguing *Git*'s usability shortcomings.

She discusses much of the *Git* terminology in her post *Confusing* Git *terminology*.

We need to disambiguate some terms:

1. GitHub fork & `git clone`
2. GitHub pull requests & `git request-pull`

[1] Amazing technologist – watch her talks!

`git request-pull` vs. GitHub Pull Requests

`git request-pull` generates a summary of changes from a branch that you want the repository maintainer to pull.

This is a *manual* communication process where you would:

1. Email the output to the repository maintainer
2. Provide a remote repository URL for them to access your changes

GitHub Pull Requests are a UI-based feature that:

1. Automates the notification process to repository maintainers
2. Creates a dedicated discussion space for code review
3. Provides visual diff tools to examine changes
4. Allows inline comments and suggested changes
5. Integrates with GitHub's permissions system for approval workflows

Pull Requests

Let’s imagine you’re in a team, and you have been charged with solving an issue.

You fork the public repository at version `v1.0` into your local, and fix the problem and commit it to your local repository.

Pull Requests

Let's imagine you're in a team, and you have been charged with solving an issue.

You fork the public repository at version `v1.0` into your local, and fix the problem and commit it to your local repository.

On the command line, you could use the command

```
git request-pull v1.0 <project_url> <local_branch_name>
```

to *request* a pull from the repository maintainer to incorporate your changes.

Pull Request Etiquette

As will all things collaborative – this is a people-problem, and technical solutions will only go so far.

You may be reviewed, or may be asked to review a contribution before merging into a project.

This is an opportunity to be direct and both give and take criticism gracefully.

Pull Request Etiquette

As will all things collaborative – this is a people-problem, and technical solutions will only go so far.

You may be reviewed, or may be asked to review a contribution before merging into a project.

This is an opportunity to be direct and both give and take criticism gracefully.

Importantly, pull request review is *not* an opportunity to be rude or insulting to people; least of all people who are contributing their time, effort, and expertise to your project.

Cloning a Repository

Instead of initializing an empty repository, you can clone an existing one.

```
git clone <url-ending-in.git>
```

This copies a GitHub repository from that url to the current directory, which may or may not be used with GitHub.

Cloning a Repository

Instead of initializing an empty repository, you can clone an existing one.

```
git clone <url-ending-in.git>
```

This copies a GitHub repository from that url to the current directory, which may or may not be used with GitHub.

You'll often be using `git clone` with source-only software that you wish to *use*. If you want to *contribute* to the software, you'll frequently use a GitHub *fork*.

Forking a Repository

On GitHub, you can **fork** a repository to create a personal copy of a repository:

1. Creates a copy of the repository on your GitHub account
2. Allows you to freely experiment with changes without affecting the original
3. Propose changes to the original repository through pull requests

Forking a Repository

On GitHub, you can **fork** a repository to create a personal copy of a repository:

1. Creates a copy of the repository on your GitHub account
2. Allows you to freely experiment with changes without affecting the original
3. Propose changes to the original repository through pull requests

To fork a repository:

1. Navigate to the repository on GitHub
2. Click the “Fork” button in the upper-right corner
3. Choose where to fork the repository (usually your account)
4. Wait for GitHub to create your copy

Forking a Repository

On GitHub, you can **fork** a repository to create a personal copy of a repository:

1. Creates a copy of the repository on your GitHub account
2. Allows you to freely experiment with changes without affecting the original
3. Propose changes to the original repository through pull requests

To fork a repository:

1. Navigate to the repository on GitHub
2. Click the “Fork” button in the upper-right corner
3. Choose where to fork the repository (usually your account)
4. Wait for GitHub to create your copy

Unlike `git clone` (which creates a local copy), forking happens entirely on GitHub and maintains a connection to the original repository.

File Management

`.gitignore`

Using *Git* for actual projects, you'll find the working directory cluttered with files.

In the case of data, this presents a security issue:

> How do we avoid publishing private files in the working directory?

[1] I recommend you include extensions foremost in the `.gitignore` file as that will prevent you from accidentally including a filename you should not.

.gitignore

Using *Git* for actual projects, you'll find the working directory cluttered with files.

In the case of data, this presents a security issue:

> How do we avoid publishing private files in the working directory?

You can exclude files from *Git* by including them in the `.gitignore` file.[1]

```
# gitignore
*.log
*.tmp
thatOneFile.txt
privateDirectory/
```

This `.gitignore` file is used to exclude log and tmp files, and the `thatOneFile.txt` file, and the `privateDirectory/` directory.

[1] I recommend you include extensions foremost in the `.gitignore` file as that will prevent you from accidentally including a filename you should not.

Large File Storage (LFS)

GitHub has a 100 MB file size limit for repositories, LFS links large files to a repo.

Large File Storage (LFS)

GitHub has a 100 MB file size limit for repositories, LFS links large files to a repo.
Typical usage:

1. Install *Git* LFS,
2. Track specific file types,
3. Add the `.gitattributes` file,
4. Use *Git* normally.

```
git lfs install
git lfs track "*.zip"    # Track zip archives
git add .gitattributes
git add archive.zip # 200MB file
git commit -m "Add large design file"
git push
```

Large File Storage (LFS)

GitHub has a 100 MB file size limit for repositories, LFS links large files to a repo. Typical usage:

1. Install *Git* LFS,
2. Track specific file types,
3. Add the `.gitattributes` file,
4. Use *Git* normally.

```
git lfs install
git lfs track "*.zip"    # Track zip archives
git add .gitattributes
git add archive.zip # 200MB file
git commit -m "Add large design file"
git push
```

LFS stores the large files separately from your main repository, replacing them with text pointers, while handling the file storage behind the scenes.

Collaborative Development

Team Management

Frequently you will work in a team to develop a piece of software[1].

GitHub offers extensive tooling for team collaboration:

- **Issues** Track bugs, enhancements, and tasks with assignees and labels
- **Projects** Kanban-style boards to visualize workflow and task progression
- **Branch protection** Enforce review requirements and prevent direct changes
- **Code owners** Automatically assign reviewers based on file patterns
- **Actions** Automate CI/CD and quality control processes
- **Discussions** Facilitate design decisions or community interaction

[1] Like you will for this coursework!

Team Management

Frequently you will work in a team to develop a piece of software[1].

Effective team management combines tools with agreed-upon conventions:

- Consistent branch naming (e.g., `feature/`, `fix/`, `docs/` prefixes)
- Pull request templates to standardize information provided
- Commit message standards for better history tracking
- Code review guidelines to ensure quality and consistency
- Code of conduct for respectful communication and collaboration

> Remember that software development is primarily a *social* activity!

[1] Like you will for this coursework!

Distributed Workflows

Any number of distributed workflows are possible.

Some common ones:

- **centralized workflow**
- **integration-manager**
- **dictator & lieutenants**

Distributed Workflows

Any number of distributed workflows are possible.

Some common ones:

- **centralized workflow** for small projects, one central repository hosts the accepted version of the code base, and every developer synchronizes their local version with it to maintain consistency.
- **integration-manager**
- **dictator & lieutenants**

Distributed Workflows

Any number of distributed workflows are possible.

Some common ones:

- **centralized workflow**
- **integration-manager** for medium-sized projects, one developer is selected to have (read-only) access to everyone's repository; this developer is then responsible for managing pull requests from everyone's repositories into the central repository.
- **dictator & lieutenants**

Distributed Workflows

Any number of distributed workflows are possible.

Some common ones:

- **centralized workflow**
- **integration-manager**
- **dictator & lieutenants** for very large projects, you might find the earlier options limiting; in this workflow, segments of the repository are under the stewardship of different lieutenants, whose work is integrated by the integration manager.

Distributed Workflows

Any number of distributed workflows are possible.

Some common ones:

- **centralized workflow**
- **integration-manager**
- **dictator & lieutenants**

Each of these places further distance between the *one true repository* and the developers doing the work, introducing more hierarchy into the organization, and thus less *productive* work per developer effort.

I.e., if you have **5** collaborators – it's less wasted effort if everyone manages consistency themselves. But if you have **100** collaborators, it may be better to delegate that effort to a handful of people.

Summary

Git Collaboration Summary

Key Points

- *Git* is a distributed version control system
- GitHub is a popular *Git* host
- Remote repositories enable collaboration
- Pull requests facilitate code review and integration

Next Steps

- Practice using *Git* with GitHub
- Explore advanced features like branching and merging
- Learn about building and testing automation (CI/CD)

There are both *strategies* and *tools* for using *Git* collaboratively; you will need to assess the needs of your project to determine how to apply them effectively.